Práctica 3 — Planificación de procesos de tiempo real

Ejercicio 3: Banco de medición

Sistemas de Tiempo Real — Ingeniería en Computación

a) Por qué espera activa y no una llamada bloqueante

Las dos alternativas modelan cosas opuestas:

	Lazo de espera activa	**`vTaskDelay()` (bloqueo)**
Estado de la tarea	*Running / Ready*	*Blocked*
CPU	la **ocupa** toda la ráfaga	la **cede** de inmediato
Qué simula	una tarea que **computa**: el C_i del enunciado	una tarea que **espera** un evento externo, E/S o un temporizador
¿Se la puede expropiar?	sí	no hay nada que expropiar

Si el cómputo se resolviera con `vTaskDelay()`, las cuatro tareas estarían bloqueadas casi todo el tiempo, la utilización real del procesador sería prácticamente nula, nunca habría dos tareas listas a la vez y **el planificador no tendría nada que arbitrar**: no habría expropiaciones, no se perderían plazos, y los cronogramas de Round Robin y de FIFO saldrían idénticos. Todo el experimento de la Parte II quedaría sin sentido.

El requisito escondido: contar trabajo, no tiempo de reloj

El enunciado agrega que una tarea expropiada debe reanudar su ráfaga *"sin perder el tiempo de cómputo ya consumido"*. Eso descarta también la forma más obvia de hacer espera activa:

```
while (millis() - t0 < C) { }     /* MAL: mide tiempo de RELOJ */
```

Ese lazo termina siempre C milisegundos después de haber empezado, la hayan expropiado o no. Si la expropian la mitad del tiempo, consume la mitad del cómputo: la tarea "trabaja" menos justo cuando el sistema está más cargado, que es lo contrario de lo que se quiere medir.

La implementación correcta cuenta **iteraciones ejecutadas**. Mientras la tarea está expropiada el contador no avanza, porque la tarea no corre; al volver, el contexto se restaura y el lazo sigue donde estaba. Con `_delay_loop_2()` de avr-libc, que consume exactamente 4 ciclos por iteración, la calibración es exacta:

$$\texttt{ITER_POR_MS} = \frac{F_{CPU}}{1000 \cdot 4} = \frac{16 \cdot 10^6}{4000} = 4000 \text{ iteraciones} = 16000 \text{ ciclos} = 1{,}000 \text{ ms}$$

b) Firmware de una sola tarea y medición

La escala de tiempos

El enunciado adopta 1 tick = 15 ms. La librería Arduino_FreeRTOS genera el tick con el **Watchdog Timer** y, aunque la constante se llame `WDTO_15MS`, el período que la propia librería calcula es

$$\texttt{portTICK_PERIOD_MS} = 2^{(0+4)} = \mathbf{16} \text{ ms}$$

Por eso el firmware expresa todo **en ticks** y no en milisegundos: así se cumple lo que importa del enunciado, que los períodos y los cómputos sean múltiplos exactos del tick. La consecuencia es que 1 unidad = 4 ticks = **64 ms reales**, no 60.

Valores teóricos para la tarea D ($C = 1$, $T = 3$ unidades)

	Teórico (enunciado, tick 15 ms)	**Teórico (tick real, 16 ms)**	**Referencia (simulación)**
Ancho de pulso	60 ms	64 ms	64,58 ms
Período	180 ms	192 ms	192,00 ms

Resultados medidos en Proteus

	Medido	**Teórico (tick real)**	**Error**
Ancho de pulso	_____	64 ms	_____
Período	_____	192 ms	_____

Por qué el ancho de pulso se pasa del valor teórico

El período sale exacto porque `xTaskDelayUntil()` se engancha a un instante absoluto y no acumula deriva. El ancho de pulso, en cambio, da algo por encima de los 64 ms. Midiendo con distintas ráfagas:

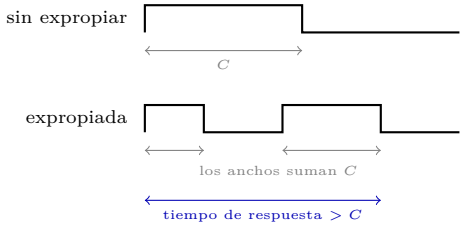
Cómputo pedido	Ancho medido	Exceso	Ticks dentro de la ráfaga
16 ms	16,149 ms	+0,149 ms	1
32 ms	32,287 ms	+0,287 ms	2
64 ms	64,580 ms	+0,580 ms	4

El exceso crece **proporcionalmente a la cantidad de interrupciones de tick que caen dentro de la ráfaga**: unos **145 μs por tick**, es decir $\approx$ 2320 ciclos a 16 MHz. Es el costo de la rutina de interrupción del planificador (salvado y restaurado del contexto completo más `xTaskIncrementTick`), que se ejecuta *dentro* de la ráfaga con el pin todavía en alto. No es un error de calibración del lazo: es **sobrecarga del sistema operativo**, y medirla es para lo que sirve este banco.

c) ¿Sigue siendo C si la expropian?

Sí, el tiempo de CPU sigue siendo C — precisamente porque el lazo cuenta iteraciones ejecutadas y no tiempo de reloj. Mientras la tarea está expropiada no ejecuta ninguna iteración, así que no gasta cómputo; cuando el planificador se lo devuelve, el contexto se restaura y el lazo continúa en el punto exacto donde quedó.

Lo que **sí cambia** es el **tiempo de respuesta**: el intervalo entre la activación y el fin de la ráfaga se alarga por la interferencia de las otras tareas. En el analizador lógico se ve así:



El pulso aparece **partido en varios tramos**. La **suma de los anchos** sigue valiendo C; lo que crece es el **lapso total**, desde el primer flanco de subida hasta el último de bajada. Esa diferencia es exactamente la **interferencia** que sufre la tarea, y es lo que se verifica en el Ejercicio 4.

Si en cambio el lazo se hubiera escrito con `millis()`, el tiempo de CPU consumido sería **menor que** C y el pulso terminaría en el mismo instante absoluto que sin expropiación: la tarea habría hecho menos trabajo del que declara.

Código

Archivo `Ejercicio3.ino`. La tabla `tareas[]` tiene una sola entrada; para el Ejercicio 4 se agregan A, B y C a ese mismo arreglo.

```
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <task.h>
#include <util/delay_basic.h>

#define TICKS_POR_UNIDAD   4
#define PRIO_PERIODICAS    2
#define PRIO_REPORTE       1
#define PERIODO_REPORTE_MS 3000

/* ---- consumo de CPU por espera activa -------------------------------
 * _delay_loop_2 consume exactamente 4 ciclos por iteracion. El lazo cuenta
 * TRABAJO EJECUTADO, no tiempo de reloj: si la tarea es expropiada en el
 * medio, el contador no avanza mientras no corre y la rafaga se reanuda
 * donde habia quedado.                                                  */
#define ITER_POR_MS   ((uint16_t)(F_CPU / 1000UL / 4UL))   /* 4000 @16MHz */

static void quemar_cpu_ms(uint16_t ms)
{
  while (ms--) _delay_loop_2(ITER_POR_MS);
}

/* ---- descripcion y estadisticas de una tarea ------------------------ */
typedef struct {
  const char *nombre;
  uint8_t     pin;            /* bit de PORTB */
  uint8_t     C_unidades;
  uint8_t     T_unidades;
  uint32_t    activaciones;
  uint32_t    perdidos;       /* plazos perdidos (D = T) */
  uint16_t    resp_max;       /* tiempo de respuesta maximo, en ticks */
  uint16_t    jitter_max;     /* jitter de activacion maximo, en ticks */
} tarea_t;

/* Ejercicio 3: una sola tarea activa. Para el Ejercicio 4 se agregan
   A, B y C a esta misma tabla.                                          */
static tarea_t tareas[] = {
  { "D", PB3, 1, 3, 0, 0, 0, 0 },
};
#define N_TAREAS (sizeof(tareas) / sizeof(tareas[0]))

/* ---- cuerpo comun de las tareas periodicas -------------------------- */
static void vTareaPeriodica(void *pv)
{
  tarea_t *t = (tarea_t *)pv;

  const TickType_t periodo  = (TickType_t)t->T_unidades * TICKS_POR_UNIDAD;
  const uint16_t   computo  = (uint16_t)t->C_unidades * TICKS_POR_UNIDAD
                                                      * portTICK_PERIOD_MS;
  TickType_t prev     = xTaskGetTickCount();
  TickType_t esperado = prev;          /* instante teorico de activacion */

  for (;;) {
    const TickType_t inicio = xTaskGetTickCount();

    /* jitter: desvio entre la activacion real y la teorica */
    const uint16_t j = (uint16_t)(inicio - esperado);
    if (j > t->jitter_max) t->jitter_max = j;

    PORTB |=  (uint8_t)(1 << t->pin);   /* arranca la rafaga */
    quemar_cpu_ms(computo);
    PORTB &= (uint8_t)~(1 << t->pin);   /* termina la rafaga */

    /* tiempo de respuesta: desde la activacion teorica hasta el fin */
    const uint16_t resp = (uint16_t)(xTaskGetTickCount() - esperado);
    if (resp > t->resp_max) t->resp_max = resp;
    if (resp > periodo)     t->perdidos++;    /* plazo = periodo */
    t->activaciones++;

    esperado += periodo;
    xTaskDelayUntil(&prev, periodo);   /* absoluto: no acumula deriva */
  }
}

/* ---- tarea de reporte (menor prioridad) ----------------------------- */
static void vReporte(void *pv)
{
  (void)pv;
  TickType_t prev = xTaskGetTickCount();

  for (;;) {
    xTaskDelayUntil(&prev, pdMS_TO_TICKS(PERIODO_REPORTE_MS));

    Serial.println(F("--- tarea  activ  perdidos  Rmax(ms)  jitter(ms) ---"));
    for (uint8_t i = 0; i < N_TAREAS; i++) {
```

```
      Serial.print(F("    "));     Serial.print(tareas[i].nombre);
      Serial.print(F("      "));   Serial.print(tareas[i].activaciones);
      Serial.print(F("        ")); Serial.print(tareas[i].perdidos);
      Serial.print(F("       "));
      Serial.print((uint32_t)tareas[i].resp_max * portTICK_PERIOD_MS);
      Serial.print(F("        "));
      Serial.println((uint32_t)tareas[i].jitter_max * portTICK_PERIOD_MS);
    }
  }
}

/* ---- arranque ------------------------------------------------------- */
void setup()
{
  Serial.begin(9600);

  DDRB  |=  (uint8_t)(_BV(PB0) | _BV(PB1) | _BV(PB2) | _BV(PB3));
  PORTB &= (uint8_t)~(_BV(PB0) | _BV(PB1) | _BV(PB2) | _BV(PB3));

  Serial.print(F("tick="));       Serial.print((uint32_t)portTICK_PERIOD_MS);
  Serial.print(F("ms  unidad="));
  Serial.print((uint32_t)TICKS_POR_UNIDAD * portTICK_PERIOD_MS);
  Serial.println(F("ms"));

  for (uint8_t i = 0; i < N_TAREAS; i++)
    xTaskCreate(vTareaPeriodica, tareas[i].nombre, 192,
                (void *)&tareas[i], PRIO_PERIODICAS, NULL);

  xTaskCreate(vReporte, "REP", 192, NULL, PRIO_REPORTE, NULL);
}

void loop() { }
```